

Podstawowe dane

Imie i nazwisko	Robert Mesek
Data	30.11.2023
Tytuł	Algorytmy geometryczne
Numer ćwiczenia	3

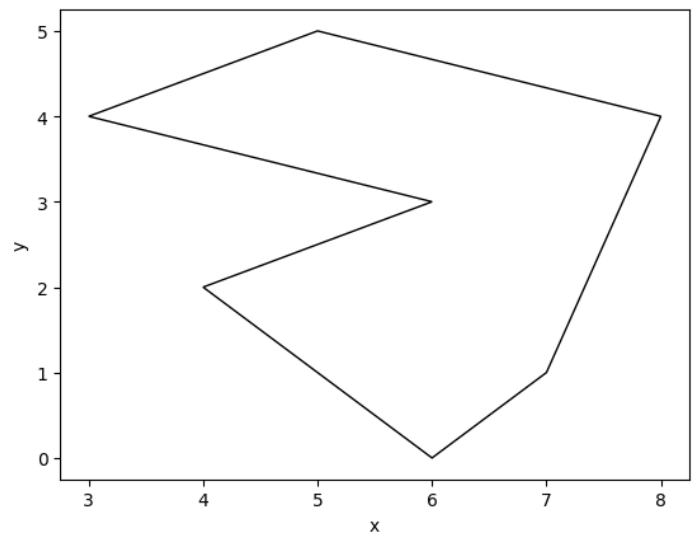
Dane techniczne

Procesor	Intel i7-1165G7 x86_64
Język	Python 3.11.5
Biblioteki	NumPy 1.26.0, Pandas 2.1.1, Matplotlib 3.8.0
System operacyjny	WSL Ubuntu 22.04 (MS Windows 10 22H2)
Środowisko	conda 23.5.2

Cel ćwiczenia

Celem ćwiczenia jest:

- sprawdzanie y -monotoniczności
- podział wierzchołków na kategorie
- triangulacja wielokąta monotonicznego

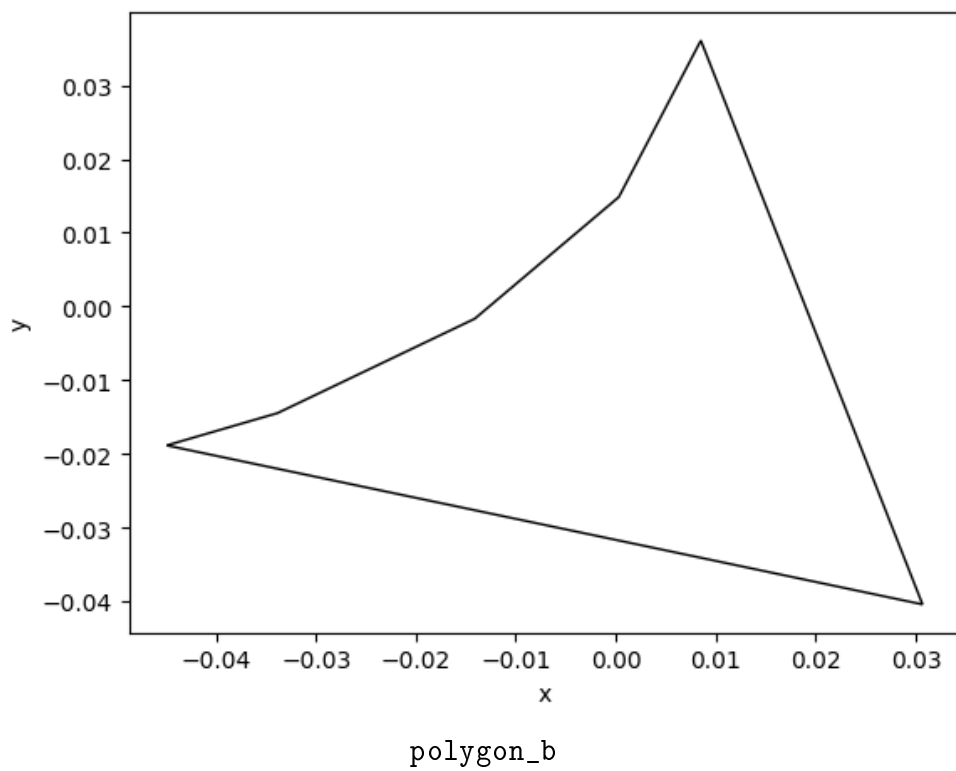
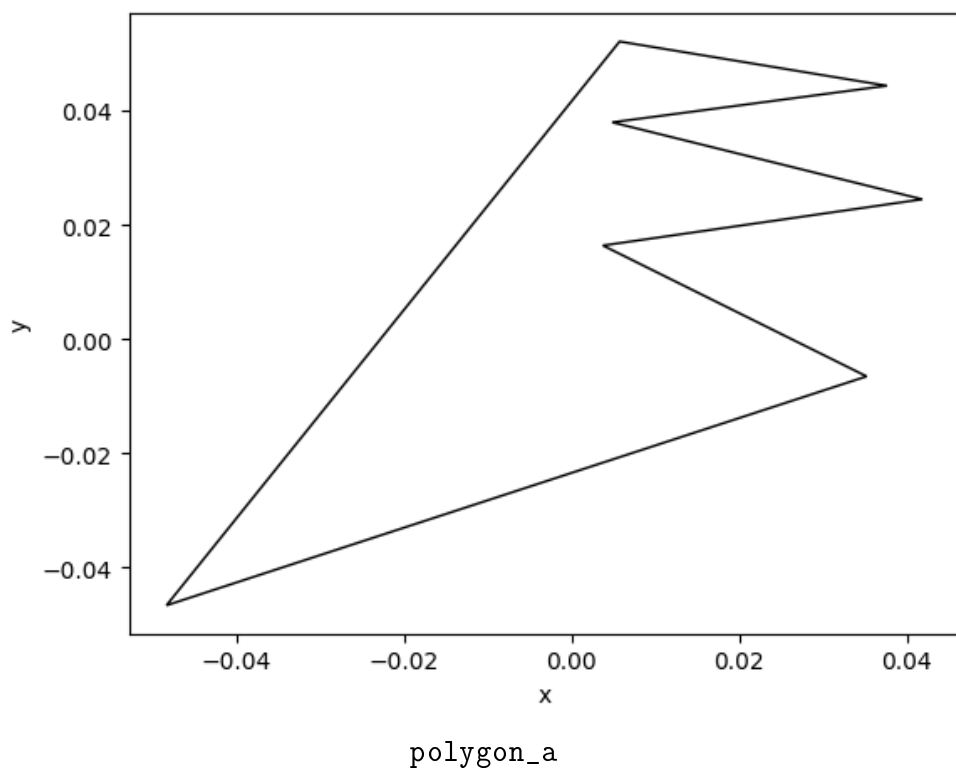


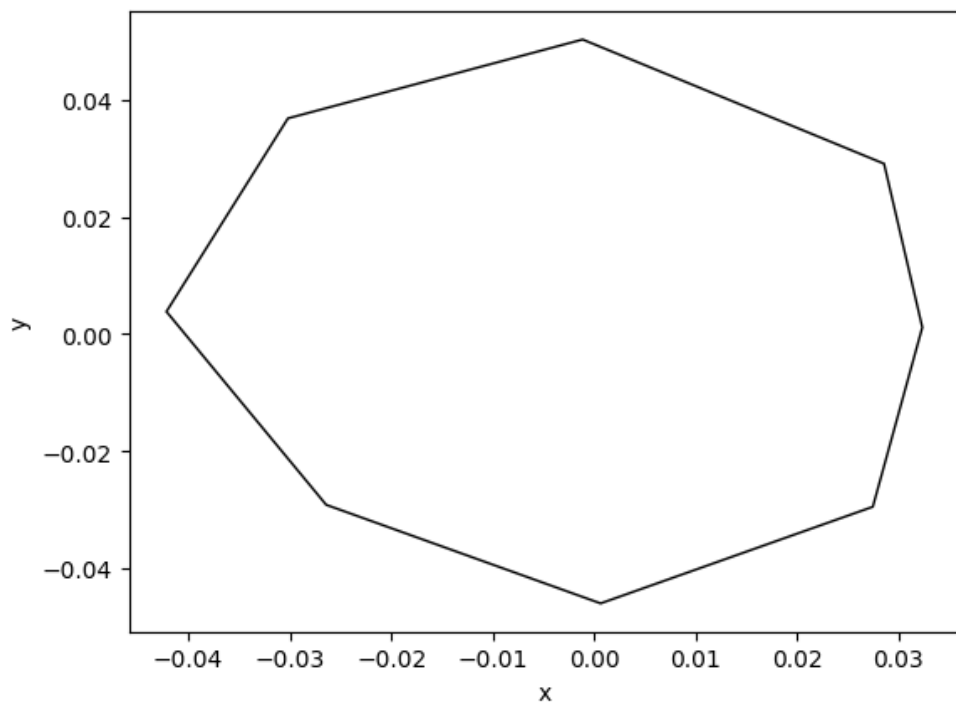
przykład: wielokąt y -monotoniczny

Zastosowania triangulacji

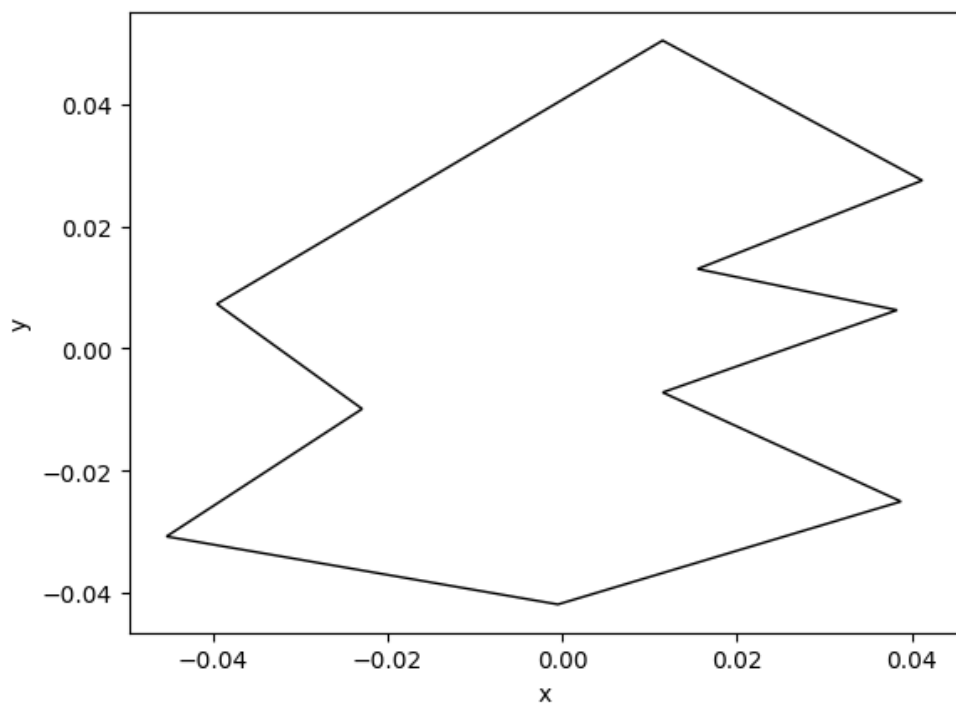
Triangulacja wielokątów w informatyce znajduje zastosowanie w grafice komputerowej, w algorytmach geometrii obliczeniowej do rozwiązywania problemów związanych z przetwarzaniem i analizą danych przestrzennych oraz analizowania sieci.

Wielokąty





`polygon_c`



`polygon_d`

Funkcje pomocnicze

Reprezentuje punkt

```
class Point:
    EPSILON = 10**-24

    def __init__(self, x, y, id=None, type=None):
        self.x = x
        self.y = y
        self.id = id

    def __repr__(self):
        return f"{self.id}. ({self.x}, {self.y})"

    def distance(self, other):
        return ((other.x - self.x) ** 2 + (other.y - self.y) ** 2) ** 0.5

    def as_tuple(self):
        return (self.x, self.y)

    def __hash__(self) -> int:
        return hash((self.x, self.y, self.id))

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Point):
            return NotImplemented
        return self.distance(other) < self.EPSILON

    def __gt__(self, other: object) -> bool:
        if not isinstance(other, Point):
            return NotImplemented
        if abs(self.x - other.x) < self.EPSILON:
            return self.y > other.y
        return self.x > other.x
```

Konwertuje listę krotek na listę punktów

```
def create_points(Q: list[tuple]):
    points = []
    for i, (x, y) in enumerate(Q):
        points.append(Point(x, y, i))
    return points
```

Oblicza wyznacznik macierzy, potrzebny do określenia orientacji punktu względem prostej

```
def det(a: Point, b: Point, c: Point):  
    """  
    Orientacja punktu "c" względem prostej "a b" obliczając wyznacznik macierzy 2x2  
    :param a: pierwszy punkt tworzący naszą prostą  
    :param b: drugi punkt tworzący naszą prostą  
    :param c: punkt, którego położenie względem prostej chcemy znaleźć  
    :return: wartość wyznacznika macierzy  
    (> 0) => Counterclockwise  
    (== 0) => Collinear  
    (< 0) => Clockwise  
    """  
    return (a.x - c.x) * (b.y - c.y) - (a.y - c.y) * (b.x - c.x)
```

Klasyfikuje punkt

```
def classify(a: Point, b: Point, c: Point, eps=Point.EPSILON):  
    """  
    :return: klasyfikacja punktu  
    starting = 0  
    closing = 1  
    connective = 2  
    separative = 3  
    correct = 4  
    """  
    # starting or separative or correct  
    if a.y < b.y and c.y < b.y:  
        # starting = 0  
        if det(a, b, c) > eps:  
            return 0  
        # separative = 3  
        elif det(a, b, c) < -eps:  
            return 3  
    # closing or connective or correct  
    elif a.y > b.y and c.y > b.y:  
        # closing = 1  
        if det(a, b, c) > eps:  
            return 1  
        # connective = 2  
        elif det(a, b, c) < -eps:  
            return 2  
    # correct = 4  
    else:  
        return 4
```

Czy wielokąt jest y-monotoniczny?

Wielokąt jest monotoniczny, gdy jego wierzchołki mogą być ułożone w taki sposób, że jedna z jego współrzędnych (na przykład współrzędna x lub y , w zależności od układu współrzędnych) zawsze rośnie lub maleje wzdłuż kolejnych wierzchołków. Innymi słowy, dla każdej pary wierzchołków wielokąta (oprócz wierzchołka startowego i końcowego), jeden z punktów ma większą (lub mniejszą) wartość danej współrzędnej niż drugi punkt.

W praktyce, wielokąt monotoniczny może być łatwiej sortowany lub przetwarzany w pewnych algorytmach geometrycznych, ponieważ istnieje pewna kolejność, w jakiej wierzchołki pojawiają się wzdłuż danej osi (np. osi x lub y). Monotoniczność może ułatwić znajdowanie przecięć linii w takim wielokącie lub wykonywanie innych operacji geometrycznych. W tym zadaniu interesuje nas monotoniczność wielokąta wzdłuż osi y .

Funkcja sprawdzająca czy wielokąt jest y-monotoniczny

```
def is_y_monotonic(polygon):
    """
    Funkcja określa czy podana figura jest y-monotoniczna.
    :param polygon: tablica krotek punktów na płaszczyźnie euklidesowej podanych
    przeciwnie do ruchu wskazówek zegara - nasz wielokąt
    :return: wartość bool - true, jeśli wielokąt jest monotoniczny i false jeśli nie jest
    """
    points = create_points(polygon)

    for i in range(len(points) - 1):
        # connective or separative
        if classify(points[i - 1], points[i], points[i + 1]) in (2, 3):
            return False
    # connective or separative
    if classify(points[-2], points[-1], points[0]) in (2, 3):
        return False
    return True
```

Podział wierzchołków na kategorie

Wierzchołki naszego wielokąta możemy podzielić na parę kategorii:

- początkowe, gdy obaj jego sąsiedzi leżą poniżej i kąt wewnętrzny ma mniej niż 180 stopni. To wierzchołki, w których zaczyna się monotoniczny spadek
- końcowe, gdy obaj jego sąsiedzi leżą powyżej i kąt wewnętrzny ma mniej niż 180 stopni. To wierzchołki, w których monotoniczność wielokąta się zmienia, czyli na przykład zaczyna się monotoniczny wzrost, jeśli wcześniej był spadek, lub na odwrót.

Wierzchołki startowe i końcowe są ważne w kontekście algorytmów przetwarzania wielokątów monotonicznych, takich jak algorytmy dziel i zwyciężaj oraz triangulacji.

- dzielący, gdy obaj jego sąsiedzi leżą poniżej i kąt wewnętrzny ma więcej niż 180 stopni. To wierzchołki, które wyznaczają przekątne (linie łączące), tworzące trójkąty podczas triangulacji.
- łączący, gdy obaj jego sąsiedzi leżą powyżej i kąt wewnętrzny ma więcej niż 180 stopni. To wierzchołki, które są połączone liniami (przekątnymi) wewnątrz wielokąta, tworząc trójkąty.

Wierzchołki łączące i dzielące odgrywają kluczową rolę w procesie triangulacji wielokątów, pozwalając na podział figury na trójkąty w sposób bezkolizyjny.

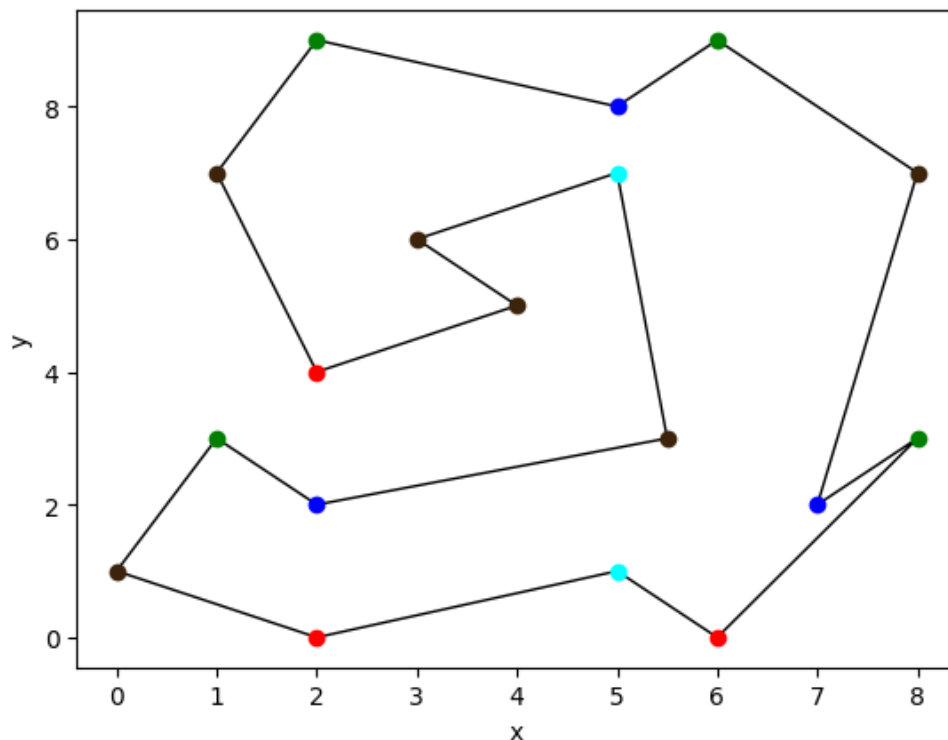
- prawidłowy, pozostałe przypadki, jeden sąsiad powyżej, drugi poniżej

Znaczenie kolorów

- Początkowy - kolor zielony
- Końcowy - kolor czerwony
- Łączący - kolor niebieski
- Dzielący - kolor cyjan
- Prawidłowy - kolor brązowy

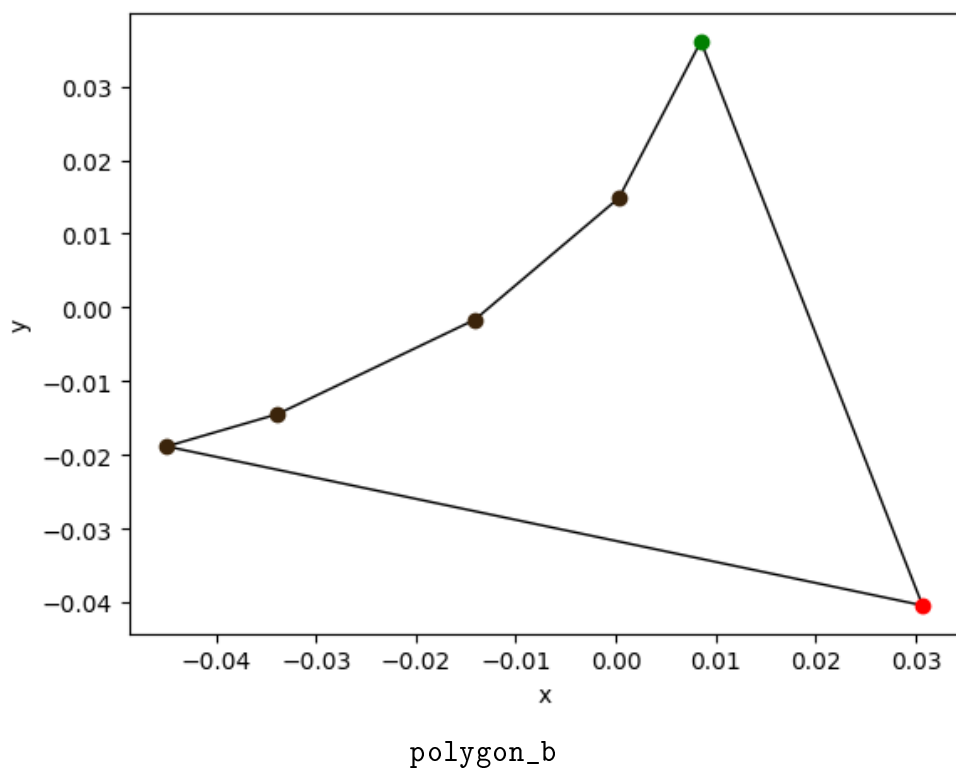
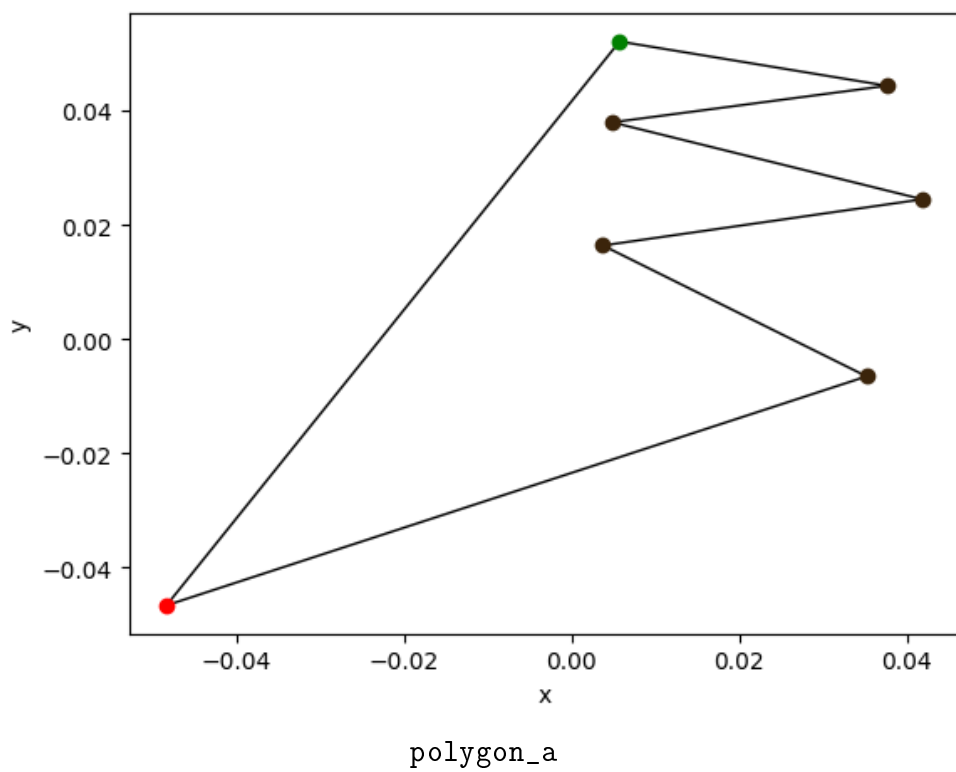
Funkcja przypisująca kolory wierzchołkom

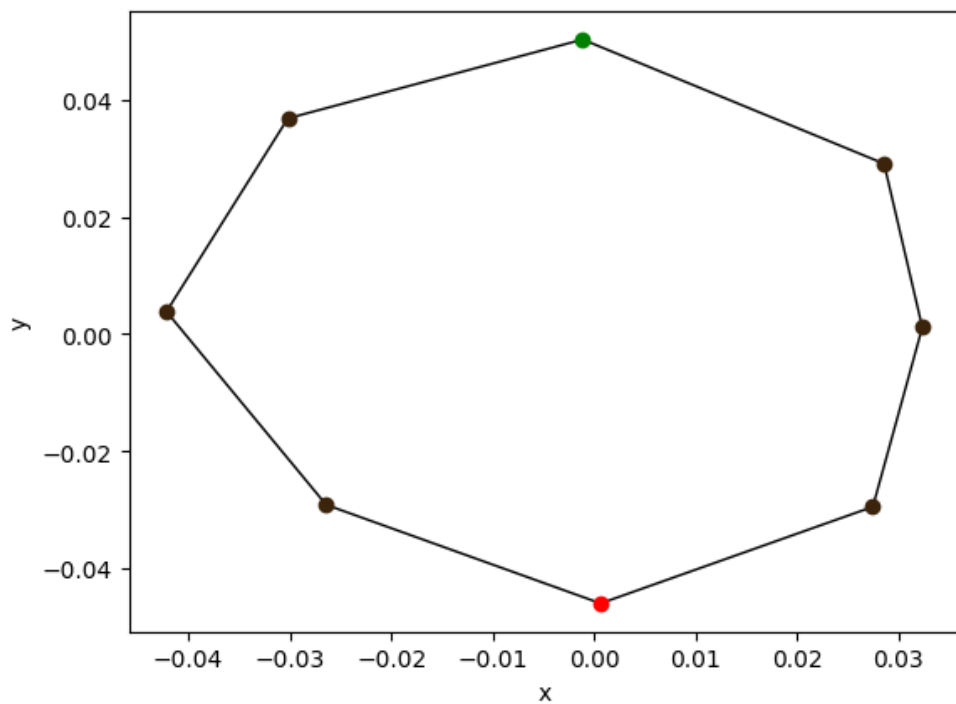
```
def color_vertex(polygon):  
    """  
    Funkcja dzieli wierzchołki na kategorie i przypisuje wierzchołkom odpowiednie numery:  
    0 - początkowy, 1 - końcowy, 2 - łączący, 3 - dzielący, 4 - prawdziwy  
    :param polygon: tablica krotek punktów na płaszczyźnie euklidesowej podanych  
    przeciwnie do ruchu wskazówek zegara - nasz wielokąt  
    :return: tablica o długości n, gdzie n = len(polygon), zawierająca cyfry z przedziału  
    0 - 4, gdzie T[i] odpowiada kategorii i-tego wierzchołka.  
    """  
    points = create_points(polygon)  
    colors = []  
  
    # all points excep first and last  
    for i in range(0, len(points) - 1):  
        colors.append(classify(points[i - 1], points[i], points[i + 1]))  
    # last point  
    colors.append(classify(points[-2], points[-1], points[0]))  
  
    return colors
```



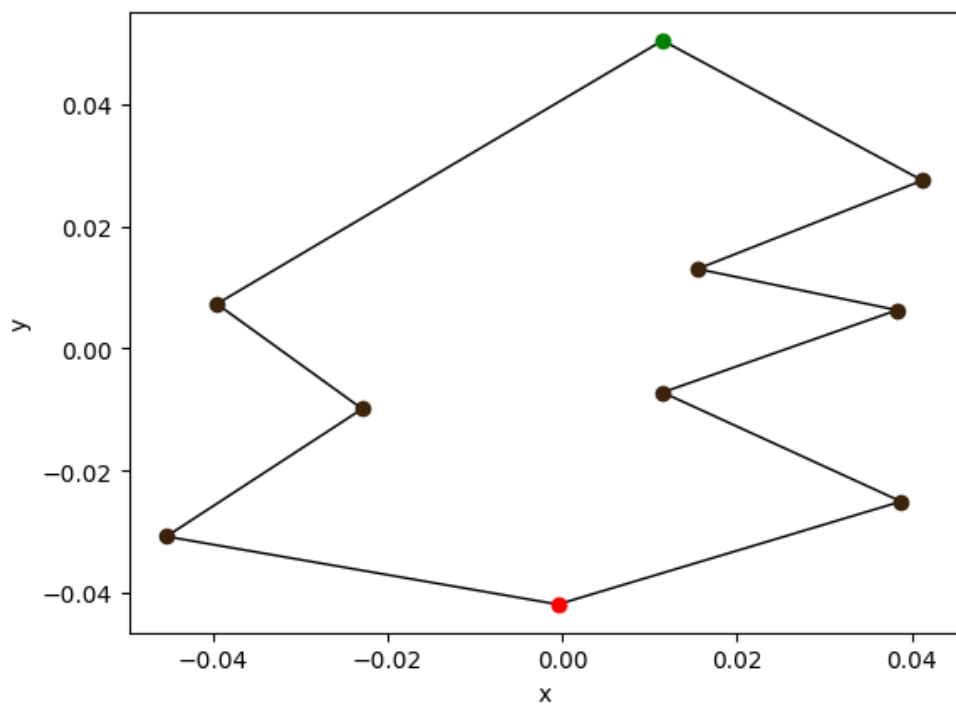
przyklad_color: Podgląd wyniku funkcji color_vertex()

Wyniki kolorowania dla naszych wielokątów





polygon_c



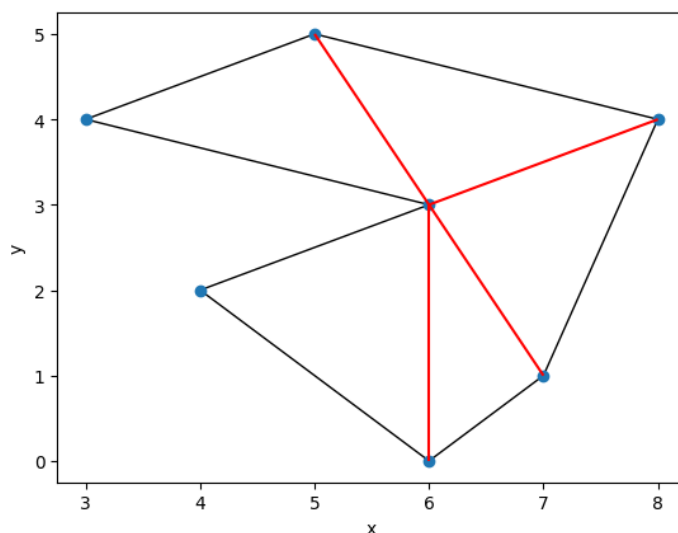
polygon_d

Wnioski:

Analizując wyniki, widzimy że wielokąty y-monotoniczne (wymagane dla triangulacji naszym algorytmem) składają się wyłącznie z punktu początkowego, końcowego oraz wierzchołków prawidłowych.

Triangulacja wielokąta monotonicznego

Triangulacja wielokąta monotonicznego to proces podziału wielokąta monotonicznego na trójkąty poprzez dodawanie przekątnych (linii łączących wierzchołki), które nie przecinają się wewnątrz.



przyklad_tri: wielokąt podzielony na trójkąty

Działanie implementowanego algorytmu (z wykładu)

- Określamy lewy i prawy łańcuch wielokąta względem kierunku monotoniczności
- Porządkujemy wierzchołki wzdłuż kierunku monotoniczności
- Wkładamy dwa pierwsze wierzchołki na stos.
- Jeśli kolejny wierzchołek należy do innego łańcucha niż wierzchołek stanowiący szczyt stosu, to możemy go połączyć ze wszystkimi wierzchołkami na stosie. Na stosie zostają dwa wierzchołki, które były „zamiatane” ostatnie.
- Jeśli kolejny wierzchołek należy do tego samego łańcucha co wierzchołek ze szczytu stosu, to analizujemy kolejne trójkąty, jakie tworzy dany wierzchołek z wierzchołkami zdejmowanymi ze stosu:
 - Jeśli trójkąt należy do wielokąta, to dodajemy przekątną i usuwamy odpowiedni wierzchołek ze szczytu stosu
 - w przeciwnym przypadku umieszczamy badane wierzchołki na stosie.

Funkcje pomocnicze dla triangulacji

Tworzy dwa **zbiory** (`set()`): łańcuch punktów z lewej oraz prawej strony wielokąta

```
def find_chains(points: list[Point]):
    right_chain = set()
    left_chain = set()
    starting = max(points, key=lambda x: x.y).id
    ending = min(points, key=lambda x: x.y).id
    i = ending
    while i != starting:
        right_chain.add(points[i])
        i = (i + 1) % len(points)
    while i != ending:
        left_chain.add(points[i])
        i = (i + 1) % len(points)
    return left_chain, right_chain
```

Sprawdza czy dwa punkty należą do tego samego łańcucha

```
def check_same_chains(left_chain, right_chain, p1: Point, p2: Point):
    if (p1 in left_chain and p2 in left_chain)
        or (p1 in right_chain and p2 in right_chain):
        return True
    return False
```

Sprawdza czy sprawdzany trójkąt znajduje się wewnątrz wielokąta

```
def triangle_in_polygon(chain, a: Point, b: Point, c: Point, epsilon=Point.EPSILON):
    if b in chain:
        return det(a, b, c) > epsilon
    else:
        return det(a, b, c) < -epsilon
```

Sprawdza czy wierzchołki są sąsiadami

```
def check_neighbours(points: list[Point], a: Point, b: Point):
    if abs(a.id - b.id) == 1 or abs(a.id - b.id) == len(points) - 1:
        return True
    return False
```

Funkcja triangulacji

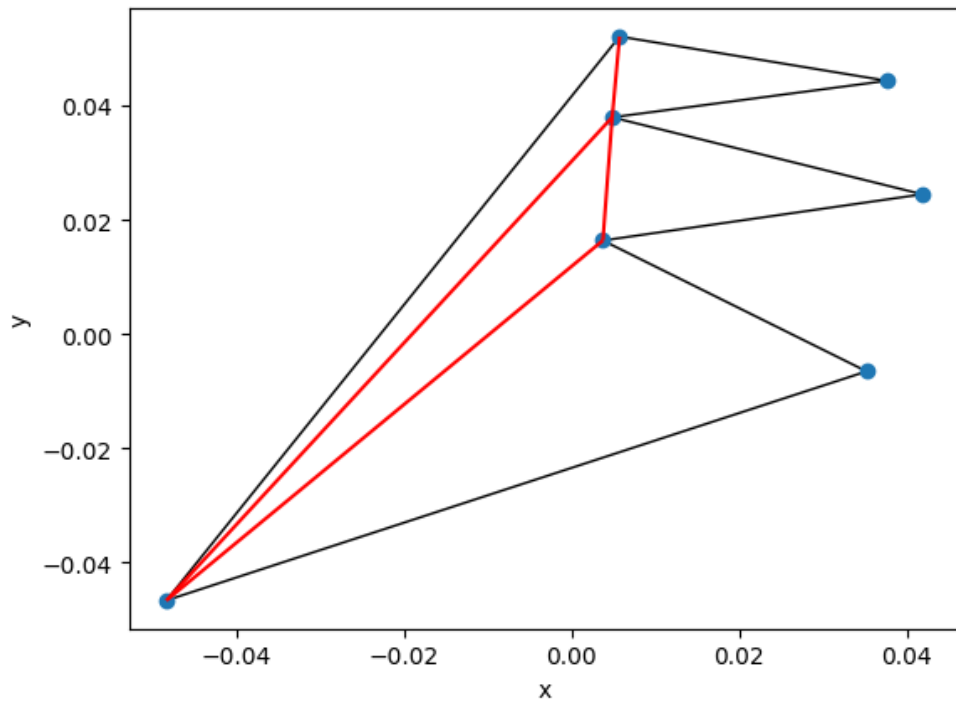
```
def triangulation(polygon):
    """
    Funkcja dokonuje triangulacji wielokąta monotonicznego.
    :param polygon: tablica krotek punktów na płaszczyźnie euklidesowej
    podanych przeciwnie do ruchu wskazówek zegara - nasz wielokąt
    :return: tablica krotek dodawanych po kolei przekątnych np: [(1,5),(2,3)],
    oznacza, że triangulacja polega na dodaniu przekątnej pomiędzy wierzchołki
    o indeksach 1 i 5 oraz 2 i 3
    """
    if not is_y_monotonic(polygon):
        raise Exception("Not y-monotonic")
    points = create_points(polygon)

    left_chain, right_chain = find_chains(points)
    points.sort(key=lambda x: x.y, reverse=True)

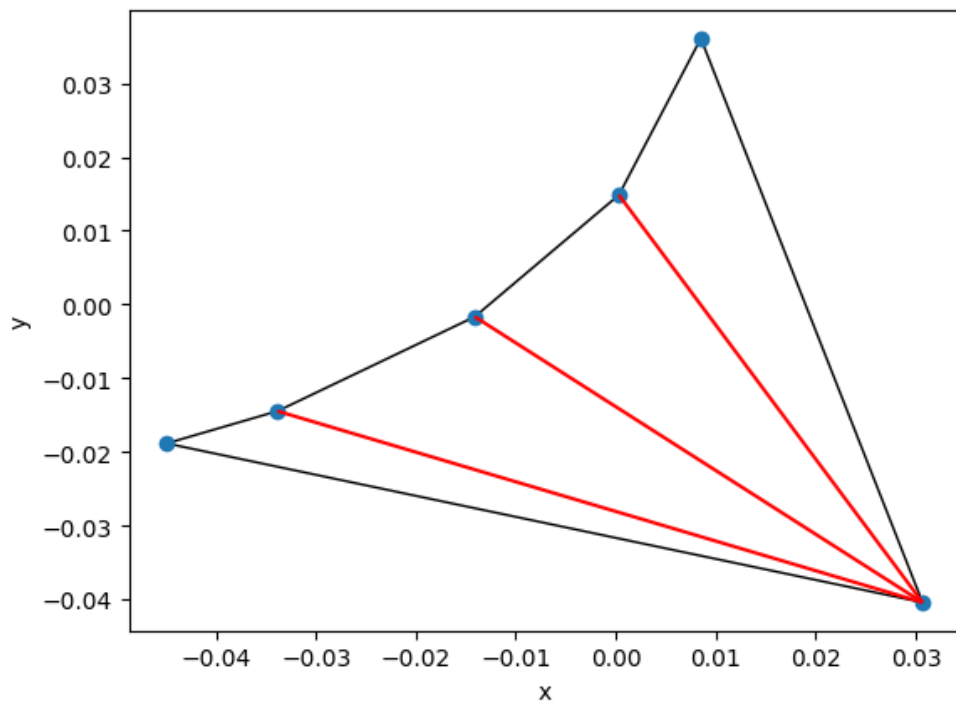
    stack = [points[0], points[1]]
    diagonals = []
    for i in range(2, len(points)):
        if not check_same_chains(left_chain, right_chain, stack[-1], points[i]):
            while len(stack) > 0:
                p = stack.pop()
                if not check_neighbours(points, p, points[i]):
                    diagonals.append((points[i].id, p.id))
            stack.append(points[i - 1])
            stack.append(points[i])
        else:
            p = stack.pop()
            while len(stack) > 0
                and triangle_in_polygon(left_chain, stack[-1], p, points[i]):
                    if not check_neighbours(points, stack[-1], points[i]):
                        diagonals.append((points[i].id, stack[-1].id))
            p = stack.pop()
            stack.append(p)
            stack.append(points[i])

    return diagonals
```

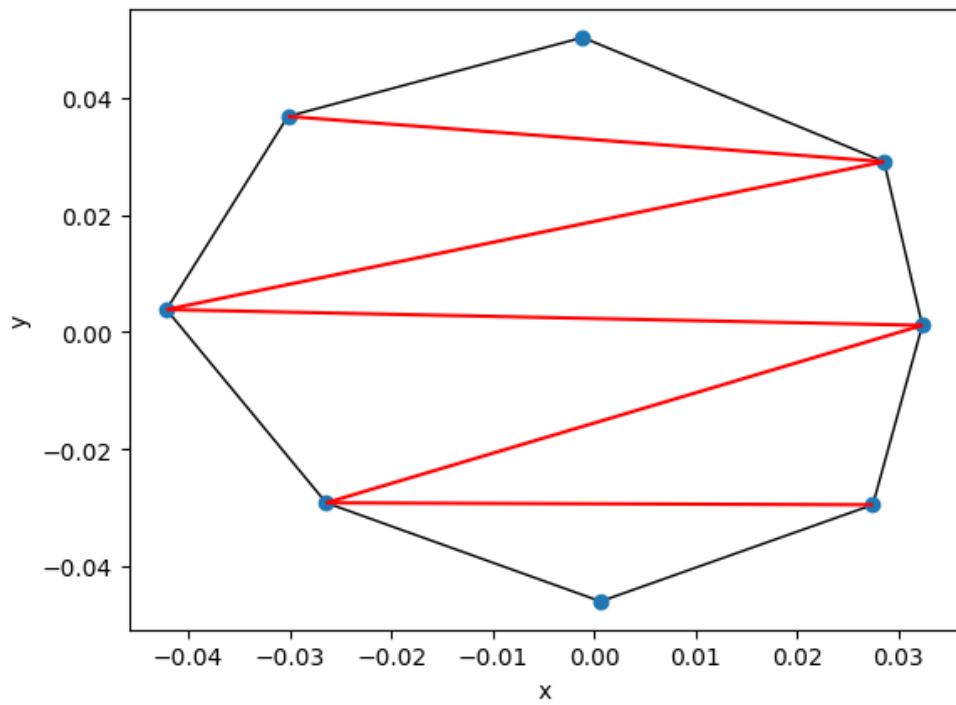
Wyniki triangulacji dla naszych wielokątów



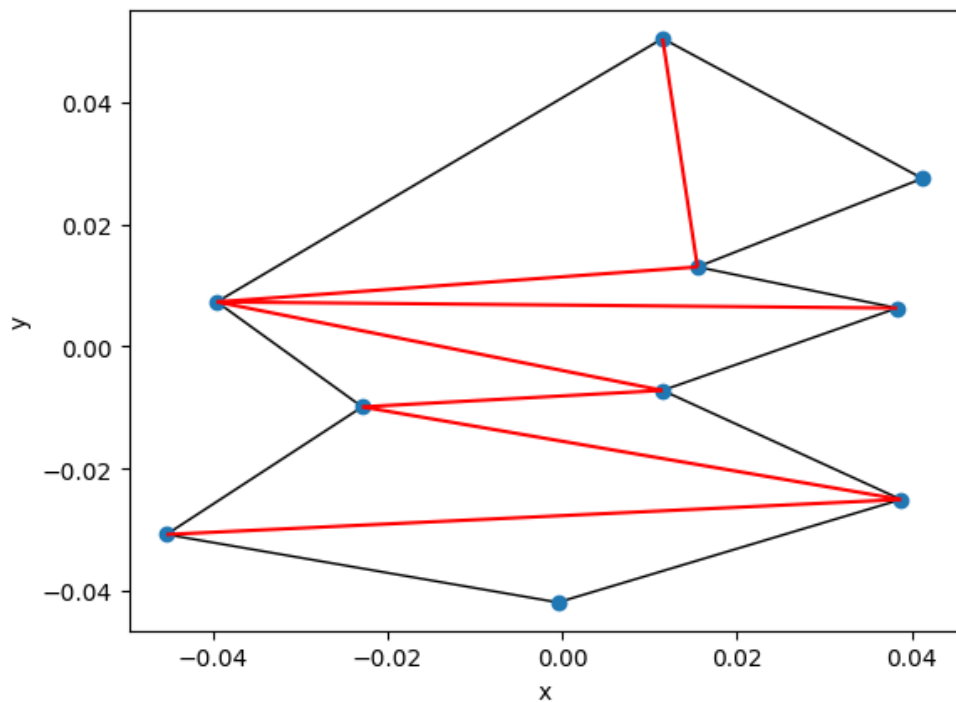
polygon_a: Poruszamy się po prawym łańcuchu



polygon_b: Poruszamy się po lewym łańcuchu
i musimy wybierać trójkąty wewnątrz wielokąta



polygon_c: Poruszamy się na przemian po łańcuchach



polygon_d: "Listek połączenie różnych przypadków

Wnioski:

Analizując wyniki, widzimy że triangulacja dla naszych wielokątów y-monotonicznych została przeprowadzona poprawnie.

Dla takich małych zbiorów wierzchołków algorytm wykonywał się momentalnie: < 70 ms.